ABSTRACT

A library management system is an essential tool for organizing and maintaining the vast amount of information associated with books, members, and transactions in any library environment. Traditional manual systems are often inefficient, prone to human error, and time-consuming when it comes to tracking book availability, issuing and returning records, and maintaining member details. As libraries grow in size and usage, the complexity of managing records increases significantly, making it difficult to ensure accuracy and quick retrieval of information. This project addresses the need for a structured and efficient system that can handle these operations seamlessly. By digitizing library processes, the system improves data integrity, reduces redundancy, and enhances user experience for both librarians and members. It also enables better tracking of borrowed books, overdue returns, and inventory management, thereby contributing to a more organized and accessible library system.

The proposed project involves the design and implementation of a relational database tailored for a library management system. The database is structured using multiple interconnected tables such as books, members, librarians, and transaction records, each defined with appropriate attributes and constraints. Primary keys are used to uniquely identify records in each table, while foreign keys establish relationships between entities, ensuring referential integrity. The design follows normalization principles to eliminate redundancy and maintain consistency across the database. Operations such as adding new books, registering members, issuing and returning books, and updating records are supported through structured queries. The system is capable of handling multiple users and transactions efficiently while maintaining accurate logs of all activities. Special attention is given to ensuring data consistency, enforcing constraints, and enabling efficient query execution, making the database robust and scalable for real-world library applications.

The project is implemented using standard database management technologies and tools commonly used in academic and practical settings. The database design is created using Entity-Relationship (ER) modeling concepts and translated into relational schemas. SQL (Structured Query Language) is used extensively for defining the database structure (DDL commands), manipulating data (DML commands), and querying information. A relational database management system such as MySQL or PostgreSQL is used to implement and manage the database. Additionally, tools like MySQL Workbench or pgAdmin may be used for visualization and administration purposes. The project may also involve the use of a simple front-end interface developed using basic programming languages or frameworks to interact with the database, though the primary focus remains on the backend database design and functionality. Overall, the chosen technology stack ensures reliability, scalability, and ease of use while adhering to fundamental database management principles.

SYSTEM ANALYSIS

A **Library Management System (LMS)** is a software solution developed to manage and automate the core functions of a library. It replaces traditional manual record-keeping with a digital system that stores information about books, members, and transactions. In a manual system, tasks such as searching for books, maintaining records, and tracking issue and return dates are slow and error-prone. The LMS addresses these issues by enabling quick access to information, reducing paperwork, and improving overall efficiency.

The proposed system provides various functionalities including adding, updating, and deleting book records, managing member registrations, issuing and returning books, and calculating fines for overdue returns. It allows users such as librarians and members to interact with the system easily through a user-friendly interface. The system also ensures better organization of data and faster processing of tasks, making library operations smoother and more reliable.

In addition to functional features, the LMS meets important non-functional requirements such as security through authentication, good performance, and data backup capabilities. The system handles essential data like book details, member information, and transaction records. From a feasibility standpoint, it is technically simple to implement, cost-effective over time, and easy to operate. Overall, the Library Management System enhances productivity, minimizes errors, and provides a structured and efficient way to manage library activities.

PROPOSED SYSTEM

The proposed Library Management System (LMS) is a web-based or desktop application designed to automate and manage all the core activities of a library in an efficient and organized manner. The system will replace the traditional manual system with a digital platform that maintains records of books, members, and transactions in a centralized database. It will provide an easy-to-use interface for both librarians and users, enabling quick access to information such as book availability, issue history, and due dates. This system reduces paperwork, minimizes human errors, and improves overall productivity.

The proposed system consists of several integrated modules. The **Admin/Librarian Module** allows the librarian to manage books (add, update, delete), maintain member records, and monitor all transactions. The **User/Member Module** enables users to search for books, view availability, and check their borrowing history. The **Circulation Module** handles issuing and returning books, automatically updates due dates, and calculates fines for late returns. The **Search Module** provides advanced search options using filters like title, author, category, or ISBN. Additionally, the system includes a **Report Generation Module** that generates useful reports such as overdue books, issued books, and inventory status.

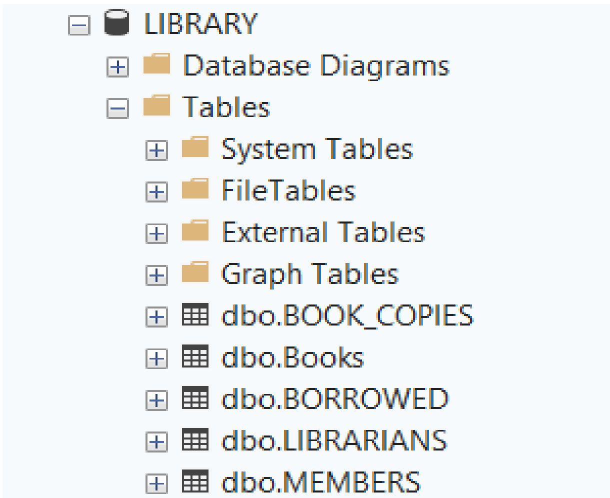
The system will incorporate essential non-functional features to ensure reliability and performance. It will include **secure authentication** (login system) to protect data, **fast processing** for handling multiple users, and **data backup and recovery mechanisms** to prevent data loss. The system is designed to be scalable, meaning it can handle increasing numbers of books and users over time. It is technically feasible using technologies like MySQL, Java/Python/PHP, and is cost-effective in the long run. Overall, the proposed LMS provides a modern, efficient, and user-friendly solution that enhances library operations and delivers better service to users.

ADVANTAGES OF A PROPOSED SYSTEM

The proposed Library Management System (LMS) offers significant advantages by automating routine library operations and reducing reliance on manual processes. Tasks such as book issuing, returning, and record maintenance become faster and more accurate, minimizing human errors. The system enables quick searching of books by title, author, or category, saving time for both librarians and users. It also provides real-time updates on book availability, helping users easily locate resources. Overall, this improves efficiency, reduces paperwork, and enhances the quality of service provided by the library.

Another major advantage of the proposed system is improved data management and security. All information related to books, members, and transactions is stored in a centralized database, making it easy to access, update, and maintain records. Features such as user authentication ensure that only authorized individuals can access sensitive data. The system can also generate useful reports, such as overdue books and fine details, which support better decision-making. Additionally, data backup and recovery options help prevent data loss, making the system reliable and suitable for long-term use.

CREATE DATABASE LIBRARY



```
CREATE TABLE Books
(BOOK_ID INT PRIMARY KEY,
TITLE VARCHAR(100),
AUTHOR VARCHAR(100),
PUBLISHER VARCHAR(100),
CATEGORY VARCHAR(100));
```

Column Name	Data Type	Allow Nulls
BOOK_ID	int	☐
TITLE	varchar(100)	☑
AUTHOR	varchar(100)	☑
PUBLISHER	varchar(100)	☑
CATEGORY	varchar(100)	☑
		☐

	BOOK_ID	TITLE	AUTHOR	PUBLISHER	CATEGORY
1	101	HARRY POTTER	RAM	RR PUBLISHERS	FANTASY
2	102	LION KING	FAYAS	ABCD PUBLISHERS	ADVENTURE
3	103	CHOTTA BHEEM	JOHN	DC BOOKS	KIDS
4	104	POKEMON	SAM	SS BOOKS	FANTASY

```
CREATE TABLE LIBRARIANS
(LIBRARIAN_ID INT PRIMARY KEY,
LIBRARIAN_NAME VARCHAR(100),
PHONE VARCHAR(15),
HIRE_DATE DATE,
SHIFT VARCHAR(20));
```

	Column Name	Data Type	Allow Nulls
🔑	MEMBER_ID	int	☐
	NAME	varchar(100)	☑
	PHONE	varchar(10)	☑
	JOIN_DATE	date	☐
	EXP_DATE	date	☑
▶		nchar(10)	☑
			☐

	MEMBER_ID	NAME	PHONE	JOIN_DATE	EXP_DATE
1	1	RAHUL	9999999999	2026-10-12	2027-10-12
2	2	ROHAN	8888888888	2026-01-01	2027-01-01
3	3	TOM	7777777777	2026-12-10	2027-12-10
4	4	SITA	6666666666	2026-06-06	2027-06-06

```
CREATE TABLE BORROWED
(BRW_ID INT PRIMARY KEY,
BOOK_ID INT,
MEMBER_ID INT,
BRW_DATE DATE,
RETURN_DATE DATE,
FOREIGN KEY (BOOK_ID)
REFERENCES books(BOOK_ID),
FOREIGN KEY (MEMBER_ID)
REFERENCES MEMBERS(MEMBER_ID));
```

Column Name	Data Type	Allow Nulls
BRW_ID	int	☐
BOOK_ID	int	☑
MEMBER_ID	int	☑
BRW_DATE	date	☑
RETURN_DATE	date	☑
		☐

	BRW_ID	BOOK_ID	MEMBER_ID	BRW_DATE	RETURN_DATE
1	1	101	1	2026-01-01	2026-02-01
2	2	102	2	2026-02-01	2026-03-01
3	3	103	3	2026-03-01	2026-04-01
4	4	104	3	2026-04-01	2026-05-01

```sql
CREATE TABLE LIBRARIANS
(LIBRARIAN_ID INT PRIMARY KEY,
LIBRARIAN_NAME VARCHAR(100),
PHONE VARCHAR(15),
HIRE_DATE DATE,
SHIFT VARCHAR(20));
```

	Column Name	Data Type	Allow Nulls
🔑	LIBRARIAN_ID	int	☐
	LIBRARIAN_NAME	varchar(100)	☑
	PHONE	varchar(15)	☑
	HIRE_DATE	date	☑
	SHIFT	varchar(20)	☑
			☐

	LIBRARIAN_ID	LIBRARIAN_NAME	PHONE	HIRE_DATE	SHIFT
1	1	ARUN	9898989898	2022-02-02	MORNING
2	2	TOMMY	9797979797	2023-01-01	AFTERNOON
3	3	SUNNY	9696969696	2024-09-09	NIGHT
4	4	DOM	9393939393	2025-10-10	MORNING

```sql
CREATE TABLE book_copies (
copy_id INT PRIMARY KEY,
book_id INT NOT NULL,
copy_number INT NOT NULL,
status VARCHAR(20) DEFAULT 'available',
purchase_date DATE,

CONSTRAINT fk_book
FOREIGN KEY (book_id)
REFERENCES books(book_id)
ON DELETE CASCADE
);
```

	Column Name	Data Type	Allow Nulls
🔑	copy_id	int	☐
	book_id	int	☐
	copy_number	int	☐
	status	varchar(20)	☑
	purchase_date	date	☑
			☐

	copy_id	book_id	copy_number	status	purchase_date
1	1	101	1	available	2023-01-15
2	2	101	2	issued	2023-02-10
3	3	102	1	available	2023-03-05
4	4	103	1	reserved	2023-04-12
5	5	104	1	lost	2023-05-20

CLASS DIAGRAM OF DATABASE